Session 4 · Building a Company Screening Engine

A workflow on live data from the U.S. Securities and Exchange Commission (SEC): your program pulls the fundamentals of 16 companies from their filings, applies two tests you control — revenue growth ≥ 8%, net margin ≥ 10% — and explains every company that passes.

In Session 2 a course-built loader fetched the data for you. Today you assemble the machinery yourself.

Plan: concepts 15' · live demo 22' · your lab 30' · debrief 8'

By the end of Session 4 you can

1. **Draw** the workflow pattern and defend the code-vs-model split
2. **Write** the code that pulls three years of revenue and profit for any ticker from its SEC filings
3. **Screen** 16 companies on two explicit tests — revenue growth of at least 8%, net margin of at least 10% — then rerun with your own thresholds
4. **Have the model explain** each surviving company in two sentences, and trace every number in those sentences back to your data
5. **Train** a model that forecasts next year's revenue, and measure its error before trusting it
6. **Express** the whole pipeline as a LangGraph workflow graph

What we are doing, and why

- **We are building a company screening engine**: fetch fundamentals for 16 companies, keep those passing two tests — revenue growth ≥ 8%, net margin ≥ 10% — and explain each survivor in audited prose.
- **New today is not the data — it is the machinery.** Session 2's loader was built for you, for one fixed peer list. Today you assemble the fetch, the filter and the audit yourself — reusable on any companies, any criteria.
- **A workflow is a plan the model cannot change**: code fetches and filters; the model only explains; code checks the explanations; you approve.
- **The finished pipeline is then rewritten as a LangGraph graph** — the form professional teams use for workflows like this.

The storyline of this session

```
Apple's raw filing record        revenue and profit, straight from EDGAR
        ↓
a fetcher for any ticker         you assemble it; 16 companies pass through
        ↓
the screen                       growth ≥ 8%, margin ≥ 10% — does Apple pass?
        ↓
rationales, audited              2 sentences per survivor, every number traced
        ↓
a revenue forecaster             next year's revenue, with its measured error
        ↓
the pipeline as a graph          LangGraph: the same steps, drawn and run
```

A workflow is a plan the model cannot change

```
INPUT → RETRIEVE → STRUCTURE → REASON → VALIDATE → HUMAN
        (code)     (code)      (model)   (code)    (you)
```

- **Code does the arithmetic**: pandas is deterministic; the model is not
- **The model reasons once**, in the middle, about a table it is *given*
- **Code audits the prose**, and a human approves: nothing is published without explicit sign-off

The data source: SEC EDGAR

- **EDGAR** (Electronic Data Gathering, Analysis and Retrieval) is the U.S. Securities and Exchange Commission's public filing database
- **What it contains**: every 10-K, 10-Q and 8-K since the mid-1990s, machine readable, free of charge, no license required
- **What it does not contain**: market prices and analyst estimates — filings are not market data

Three complications you will meet in the lab, because **companies tag their own accounts**:

1. The same item **changes names** across years
2. Foreign filers report in **local currency**
3. Most pharma reports **no operating income line at all**

Demo: the workflow end to end

A company memo built live from the filings. Two things to observe:

- **The data gaps**: the memo lists what it could *not* know from its inputs — the blind spots are declared, not hidden
- **A planted error**: one figure in the memo is altered by hand, and the audit finds it

```
audit: every figure in the prose  →  checked against inputs  →  the altered
                                                                figure is flagged
```

The libraries in this session

Library	Role here	Standing
`requests`	fetches SEC filings over HTTP	the standard HTTP library
`pandas`	the deterministic screen	the industry standard for data work
`numpy`	trains the least-squares forecaster	the numerical foundation of Python
`pydantic`	typed, validated model output	the industry standard for validation
LangGraph	the pipeline as an explicit graph	a leading framework for AI workflows, from the LangChain ecosystem

How the labs work

Every exercise sits between two markers. **You fill the gaps. Nothing else changes.**

```
### START CODE HERE ###
mask = (df[None] >= min_growth) & (df[None] >= min_margin)
### END CODE HERE ###
```

- `None` → replace with the correct column, value or variable
- `[QUESTION IN CAPITALS]` → replace with the text the bracket asks for
- **Everything else is given.** Do not rewrite it.
- Then run the ✅ **check cell** directly below. Green means correct: continue.

Stuck for two minutes? Select the lines, press `Option+K` (`Alt+K` on Windows), and ask the ✱ panel.

Your lab · notebook 04 · 30 minutes

A screening engine: Apple's peer universe + pharma + industrials, live from EDGAR

- **Exercise 1**: fetch 3 years of fundamentals for 16 companies, tolerating failed tickers
- **Exercise 2**: the filter — growth ≥ 8% and margin ≥ 10%, **pandas decides which companies pass, never the model**. Note what happens to Apple
- **Exercise 3**: one grounded rationale per survivor + numeric audit
- **Exercise 4**: train a revenue forecaster and measure its error on a held-out year
- **Exercise 5**: wire the same pipeline as a LangGraph graph — the graph prints its own diagram

Two questions for the debrief: why net margin, not operating margin?

Key takeaway

The value of this session is repeatability: a screen you can rerun any morning, with criteria you control and prose your code audits.

Next session: the model plans its own steps, under controls you define — and your finished tool goes on GitHub.